

# Documentación - Escáner de Red

## 1. Para qué sirve el programa

El programa desarrollado tiene como finalidad realizar un escaneo de direcciones IP dentro de un rango definido por el usuario. Permite verificar la conectividad de equipos en la red, mostrando información básica como la dirección IP, el nombre del host, el estado de conexión y el tiempo de respuesta en milisegundos. Además, se integraron parámetros adicionales de diagnóstico (**-r**, **-s**, **-a**) que aportan información complementaria sobre las rutas, las estadísticas y las conexiones activas.

Su propósito es ofrecer una herramienta sencilla de diagnóstico de red, orientada a usuarios técnicos y administradores. Su propósito es ofrecer una herramienta sencilla de diagnóstico de red, orientada a usuarios técnicos y administradores.

## 2. Cómo está armado el sistema

El sistema se compone de tres módulos principales: la interfaz gráfica (VentanaPrincipal), el núcleo de validación (ValidadorIP) y la ejecución del escaneo (EscanerRed). La clase Main funciona como punto de entrada al programa. La arquitectura sigue un patrón modular donde cada clase cumple un rol específico. El siguiente diagrama esquematiza la relación entre los componentes:

| Main             | VentanaPrincipal | ValidadorIP               | EscanerRed                |
|------------------|------------------|---------------------------|---------------------------|
| Punto de entrada | Interfaz gráfica | Validación de direcciones | Procesamiento del escaneo |

## 3.

### Qué métodos se usaron y por qué

Se utilizaron métodos orientados a objetos en Java, destacando los siguientes:

SwingUtilities.invokeLater() para asegurar que la interfaz gráfica se ejecute en el hilo de eventos adecuado; DefaultTableModel para manejar dinámicamente los resultados en la tabla; JProgressBar para proporcionar retroalimentación visual al usuario durante el escaneo; y validación mediante división de cadenas y manejo de excepciones en la clase ValidadorIP. Estos métodos fueron seleccionados por su estabilidad y facilidad de integración con Swing.

## **4. Por qué se eligieron ciertas tecnologías**

Se eligió Java como lenguaje de desarrollo por su portabilidad, robustez y amplia disponibilidad de bibliotecas para interfaces gráficas (Swing) y redes. Swing fue seleccionado por su facilidad para crear interfaces de escritorio ligeras sin dependencias externas. El uso de hilos (Thread en EscanerRed) asegura que la interfaz no se bloquee durante la ejecución de tareas pesadas, manteniendo la experiencia de usuario fluida. Procesos externos (ping, nslookup, -r, -s, -p udp) fueron necesarios porque ofrecen un diagnóstico real del sistema operativo, evitando tener que programar todo el análisis de red manualmente.

## **5. Qué problemas aparecieron y cómo se solucionaron**

Durante el desarrollo surgieron desafíos relacionados con la validación de IPs y la concurrencia. Inicialmente, direcciones IP mal formateadas provocaban errores en el proceso de escaneo; esto se resolvió implementando la clase ValidadorIP. Otro problema detectado fue el congelamiento de la interfaz cuando se realizaban múltiples pings; para solucionarlo, se implementó la ejecución del escaneo en un hilo separado, evitando bloqueos en la ventana principal. Compatibilidad de comandos → En lugar de **netstat**, se incorporaron los parámetros **-r**, **-s** y **-p udp** de Windows, que ofrecen información de rutas, estadísticas y conexiones activas de manera más confiable.

## **6. Qué se podría mejorar en el futuro**

Entre las posibles mejoras futuras se encuentran: la incorporación de exportación de resultados a formatos como CSV o PDF; la integración de más parámetros de diagnóstico de red (puertos abiertos, servicios disponibles); la implementación de un historial de escaneos previos; y una interfaz más moderna mediante JavaFX o frameworks multiplataforma. Además, se podrían añadir opciones de configuración avanzadas para usuarios con mayor conocimiento técnico.